German University in Cairo

Operating Systems Simulator Handbook

Team Windows XP

Project Architecture, GUI Guide, Output Explanation, and Evaluation Notes

GUC Operating Systems Simulator Handbook

Cover

- University: German University in Cairo
- Course: Operating Systems
- Team Name: Windows XP
- Project Type: Operating System Simulator in Java
- Main Features: Interpreter, Memory, PCB, Mutexes, Scheduling, Swapping, GUI

---

1. Why This Document Exists

This handbook is written for teammates who did not deeply read the original project description and need to understand the whole project clearly and quickly.

It explains:

- what the project is trying to simulate
- how the code is organized
- how memory is represented
- how scheduling works
- how mutexes and blocking work
- how swapping works
- what the GUI buttons do

- how to read the output during evaluation
- how to explain the project professionally in front of an evaluator

This is not only a technical note. It is also a presentation guide.

---

2. Big Picture: What Are We Building?

We are building a simplified simulation of an operating system.

A real operating system is responsible for:

- loading programs
- turning them into processes
- allocating memory to them
- deciding which process runs next
- protecting shared resources
- handling input and output
- moving data in and out of memory when memory becomes full

Our project simulates all of those ideas in a controlled Java environment.

Instead of running real machine code, we read simple text files such as `Program1.txt`, `Program2.txt`, and `Program3.txt`. Each line in those files represents one instruction. Once the file is loaded into the simulator, it becomes a process.

So the system is doing the following cycle:

1. read program file
2. create process
3. place process in memory
4. schedule process
5. execute one instruction at a time
6. update queues, memory, and process state
7. swap when needed
8. continue until all processes finish

---

3. Core Concepts You Must Understand First

3.1 Program vs Process

A **program** is just a file that contains instructions.

A **process** is a program after the operating system loads it and starts managing it.

In our project:

- `Program1.txt` is only text before loading
- once loaded, it becomes `P1`
- `P1` then has a PCB, memory boundaries, state, and a program counter

3.2 Process State

Each process can be in one of these states:

- `READY`: waiting to run
- `RUNNING`: currently executing
- `BLOCKED`: waiting for a resource
- `FINISHED`: completed execution

This state changes throughout the simulation, and these changes are printed in the output and tracked in the PCB.

3.3 Program Counter

The program counter, usually called `PC`, is the index of the next instruction to execute.

Example:

- if `PC = 0`, the process will execute the first instruction
- after executing one instruction, the `PC` becomes `1`

The `PC` is stored both in the Java process object and inside memory as part of the PCB data.

---

4. Understanding the Memory Model

4.1 Why Memory Exists in This Project

The project description requires us to simulate a fixed-size memory, not just use Java objects invisibly.

That means we need a memory data structure that visibly holds:

- process instructions
- variables

- PCB information

This helps us demonstrate memory management as an operating system concept.

4.2 Memory Size

The memory size is fixed at:

- `40` memory words

Each word can hold one key-value pair.

In code, each memory word is represented by a `MemoryWord` object:

- `key`
- `value`

Examples:

- `[code_0: semWait userInput]`
- `[var_x: 10]`
- `[pcb_state: RUNNING]`

4.3 What Is a Memory Slot?

A **memory slot** or **memory word** is one position in the memory array.

Example:

- slot `0` might contain the first instruction
- slot `7` might contain variable `x`
- slot `10` might contain PCB data like process ID

When the output prints:

```
0 -> [code_0: semWait userInput]
1 -> [code_1: assign x input]
...
7 -> [var_x: 10]
10 -> [pcb_id: 1]
```

that means those specific memory positions are currently occupied by parts of one process.

4.4 How a Process Is Stored in Memory

Each process needs:

- all its instruction lines
- `3` variable slots
- `4` PCB slots

So if a process has:

- `7` instructions

Then total process size is:

- `7 instructions + 3 variables + 4 PCB = 14 words`

That is exactly why Program 1 and Program 2 each consume 14 memory words.

Program 3 has `9` instructions, so it needs:

- `9 + 3 + 4 = 16` words

4.5 Why Swapping Happens

If memory contains:

- Program 1 = 14 words
- Program 2 = 14 words

Then used memory = `28`

If Program 3 arrives and needs `16` words:

- total would become `44`
- but memory has only `40`

So memory becomes full and one process must be swapped out.

This is why the simulator writes a process to disk and later restores it.

---

5. PCB: Process Control Block

The PCB is the operating system's record about a process.

In this project, the PCB stores:

- Process ID
- Process State
- Program Counter

- Memory Boundaries

Example in output:

```
10 -> [pcb_id: 1]
11 -> [pcb_state: RUNNING]
12 -> [pcb_pc: 3]
13 -> [pcb_bounds: 0-13]
```

This means:

- the process ID is `1`
- it is currently `RUNNING`
- the next instruction index is `3`
- the process occupies memory from slot `0` to slot `13`

---

6. Interpreter: How Instructions Actually Run

The interpreter is the part that reads the current instruction and decides what to do.

Example instruction:

```
assign x input
```

The interpreter understands that:

- variable name = `x`
- source of value = `input`

So it asks for input, then stores the result in the process variable area.

Another example:

```
assign b readFile a
```

This means:

- first resolve variable `a`
- treat its value as a filename
- read that file
- store the result inside variable `b`

This interpreter is one of the most important parts of the project because it connects the text programs to the simulated OS behavior.

---

7. System Calls

System calls are the interface between the process and the operating system.

In this project, the system calls include:

- reading from file
- writing to file
- printing to screen
- taking user input
- reading from memory
- writing to memory

When the process wants something, it does not manipulate the outside world directly. Instead, the interpreter routes that request through `SystemCalls.java`.

That is why the architecture still feels like an OS simulation rather than random Java code.

---

8. Mutexes and Mutual Exclusion

8.1 Why Mutexes Matter

Two processes must not use the same critical resource at the same time.

The project requires three protected resources:

- `userInput`
- `userOutput`
- `file`

8.2 What `semWait` Does

When a process executes:

```
semWait file
```

it is asking:

- "Can I lock the file resource?"

If the file is free:

- the process acquires it

If the file is busy:

- the process becomes `BLOCKED`
- it joins the blocked queue of that resource
- it also joins the general blocked queue

8.3 What `semSignal` Does

When a process finishes using a resource, it executes:

```
semSignal file
```

This:

- releases the resource
- wakes up the next blocked process if one exists

That unblocked process becomes `READY` again.

---

9. Scheduling

Scheduling means choosing which ready process runs next.

Our project supports:

- HRRN
- Round Robin
- Multi-Level Feedback Queue (MLFQ)

9.1 HRRN

HRRN means Highest Response Ratio Next.

Formula:

```
Response Ratio = (Waiting Time + Burst Time) / Burst Time
```

This gives priority to processes that have:

- waited longer
- or are shorter and efficient to run

The simulator calculates this ratio and chooses the highest one.

Example:

```
P2 -> (W:6 + S:7) / S:7 = 1.857
P3 -> (W:3 + S:9) / S:9 = 1.333
```

So `P2` wins because its ratio is higher.

9.2 Round Robin

Round Robin uses:

- a ready queue
- a quantum

Each process gets a fixed number of instructions, then if it still has more work:

- it returns to the end of the queue

In our project the required default quantum is:

- `2`

This is useful for fairness and visible switching behavior.

9.3 Multi-Level Feedback Queue (MLFQ)

MLFQ is a dynamic priority scheduler that uses multiple ready queues instead of one single queue.

In our implementation, it follows the project description:

- `4` priority queues
- every new process starts in `RQ0`
- queue quantums follow `2^i`
- so the queue quantums are:
- `RQ0 = 1`
- `RQ1 = 2`
- `RQ2 = 4`
- `RQ3 = 8`
- the scheduler always chooses from the highest-priority non-empty queue
- the last queue behaves like Round Robin

The most important rule is this:

- if a process uses its full quantum, it is demoted to the next lower queue
- if it blocks before finishing its quantum, it is not demoted because it did not consume the whole slice

- if a higher-priority process becomes ready, it can preempt a lower-priority running process

This means shorter and newer jobs tend to stay in the upper queues, while longer jobs drift downward gradually.

That is exactly why MLFQ is useful when the system does not know burst lengths in advance.

You can think of MLFQ as a filter system:

- CPU-light processes stay near the top
- CPU-hungry processes gradually sink lower
- the scheduler always gives the upper queues the first chance to run

The 4-Queue Hierarchy

Our assignment-specific MLFQ uses four queues:

- `Q0`: highest priority, quantum `1`
- `Q1`: next priority, quantum `2`
- `Q2`: next priority, quantum `4`
- `Q3`: lowest priority, quantum `8`

So the quantums follow the formula:

- `2^i`

where `i` is the queue index starting from `0`.

The Four Core Rules

The implementation follows these rules:

1. Every new process enters at `Q0`.
2. The scheduler always chooses the highest-priority non-empty queue first.
3. If a process uses its full quantum and still has work left, it is demoted one level.
4. If a process finishes or blocks before consuming its entire quantum, it is not demoted by that event.

There is also one important preemption rule:

- if a higher-priority process becomes ready while a lower-priority process is running, the lower-priority process is preempted immediately

This is what makes MLFQ different from a simple single-queue Round Robin system.

Example of Demotion

Suppose process `P1` arrives with burst time `12`.

It would behave like this:

1. In `Q0`, it runs for `1` unit, then is demoted. Remaining work becomes `11`.
2. In `Q1`, it runs for `2` units, then is demoted. Remaining work becomes `9`.
3. In `Q2`, it runs for `4` units, then is demoted. Remaining work becomes `5`.
4. In `Q3`, it can run up to `8` units, so it finishes there.

This is exactly the idea behind the lecture example: long jobs drift downward while short or newly arrived jobs keep getting faster access near the top.

Important project note:

- the original project description explicitly says MLFQ is graded as part of the assignment grade worth `5%`, not the main project grade

We still implemented it fully so the simulator now supports all three scheduling modes in one product.

---

10. Swapping Explained Clearly

Swapping is what the OS does when memory is not enough for all active processes at once.

In this project:

1. a process arrives
2. memory does not have enough free space
3. one process is selected as a victim
4. its memory image is written into a disk file such as:
- `process_2_disk.txt`
5. its memory cells are cleared
6. the new process gets loaded
7. later, when the swapped-out process is needed again, it is restored into memory

Important detail:

Swapped-out processes are **not removed from scheduling**. They still logically exist. They are only temporarily removed from RAM.

This matches the project description closely.

---

11. How To Read the Output

This section is extremely important for evaluation.

11.1 Simulation Start

Example:

```
===== OS SIMULATION START =====
Scheduler: HRRN
```

This tells us:

- simulation began
- active scheduler mode is HRRN

11.2 Clock

Example:

```
>>> CLOCK: 4 <<<
```

This means we are currently in time step `4`.

Every clock cycle is one simulation step.

11.3 Arrival

Example:

```
[ARRIVAL] P3 arrived at time 4
```

This means process `P3` was just created and entered the system at time `4`.

11.4 Scheduling Decision

Example:

```
[EVENT] Chosen
READY Queue: P3
RUNNING: P2
```

This means:

- scheduler just selected a process
- `P2` is now running
- `P3` is still waiting in the ready queue

11.5 Instruction Execution

Example:

```
[RUNNING] P2
[INSTRUCTION] writeFile a b
```

This means process `P2` is currently executing the `writeFile` instruction.

11.6 Memory Output

Example:

```
7 -> [var_a: myfile.txt]
8 -> [var_b: hello]
```

This means:

- variable `a` was stored in slot `7`
- variable `b` was stored in slot `8`

11.7 Swap Output

Example:

```
[SWAP] P2 swapped OUT
```

This means process `P2` was moved from RAM to disk.

Example:

```
[SWAP] P2 swapped IN
```

This means process `P2` was restored from disk into memory.

11.9 MLFQ Queue Output

When the scheduler is in MLFQ mode, the trace prints lines like:

```
RQ0 (q=1): P2
RQ1 (q=2): None
RQ2 (q=4): P1
RQ3 (q=8): None
```

This means:

- `P2` is waiting in the top queue
- `P1` is waiting in a lower queue

- the scheduler must choose from `RQ0` before it considers `RQ2`

If the trace says:

```
[EVENT] MLFQ Quantum Expired
```

that means the running process used its full time slice and was demoted.

If the trace says:

```
[EVENT] Preempted By Higher Queue
```

that means a higher-priority ready process appeared and interrupted the lower-priority running one.

11.8 Process Finished

Example:

```
[SYSTEM] P1 finished.
```

This means:

- process `P1` has no more instructions
- its memory can be freed

---

12. Understanding the GUI

12.1 Purpose of the GUI

The GUI is meant to:

- make the system easier to demo
- let users choose scheduler mode
- fill inputs quickly
- run repeatable examples
- view the execution trace without using raw console mode
- support evaluation with a cleaner experience

12.2 Why the Design Looks Like This

The GUI uses:

- black
- red

- gold

to reflect German flag colors and visually connect the project to GUC.

The team name `Windows XP` is also shown so the demo looks personalized and professional.

---

13. GUI Buttons and Controls

13.1 Scheduler Setup

Algorithm

Lets the user choose:

- `HRRN`
- `RR`
- `MLFQ`

RR Quantum

Used only if Round Robin is selected.

It defines how many instructions the process can execute before being sent back to the end of the ready queue.

When `MLFQ` is selected, this field is disabled because MLFQ uses fixed built-in quantums:

- `1`
- `2`
- `4`
- `8`

13.2 Program Inputs

Program 1 Start

The first number for `printFromTo`.

Program 1 End

The second number for `printFromTo`.

Program 2 File Name

The file that Program 2 will create or write to.

Program 2 File Data

The text content written into the file.

Program 3 File Name

The file that Program 3 will read and print.

13.3 Buttons

Run Simulation

Runs the entire simulator using the current values in the GUI.

Use this after you confirm:

- scheduler mode
- RR quantum if needed
- all program inputs

Load HRRN Demo

Loads a clean and correct ready-made scenario into the currently empty fields:

- Program 1 prints values between numbers
- Program 2 writes `hello` to `myfile.txt`
- Program 3 reads `myfile.txt`

Best for evaluations.

Load RR Demo

Loads a preset aimed more at showing:

- Round Robin queue rotation
- quantum-based preemption
- clearer context switching
- file writing and file reading in the same run

Load MLFQ Demo

Loads a preset aimed more at demonstrating:

- MLFQ behavior
- switching

- swapping
- final file handoff from Program 2 to Program 3

Clear Output

Clears the trace area and resets the visible metrics.

Team Windows XP Tab

This is a separate presentation tab in the GUI, not a small button inside the controls area.

It shows:

- university identity
- team name
- each member's full name
- student email
- student ID
- tutorial or section when available

Use this tab at the beginning of the demo to introduce the team professionally.

Important note:

The simulator now starts with empty input boxes by default. This gives the GUI a cleaner and more professional appearance. Sample values are only inserted when the preset buttons are used.

---

14. What the Summary Cards Mean

Status

Shows whether the simulator is:

- Idle
- Running
- Completed
- Failed

Scheduler

Shows the active scheduler mode:

- HRRN
- RR

- MLFQ

Outputs

Counts how many `[OUTPUT]` lines were printed.

Swaps

Counts how many swap events happened.

Finished

Counts how many processes completed.

---

15. Example Walkthrough of a Correct Demo

Use these inputs:

- Program 1 Start = `10`
- Program 1 End = `15`
- Program 2 File Name = `myfile.txt`
- Program 2 File Data = `hello`
- Program 3 File Name = `myfile.txt`

What will happen:

1. `P1` arrives and begins execution.
2. `P2` arrives later and waits.
3. `P3` arrives when memory is too full.
4. `P2` gets swapped out.
5. `P1` finishes.
6. `P2` is swapped back in and writes `hello` to `myfile.txt`.
7. `P2` finishes.
8. `P3` runs, reads `myfile.txt`, stores `hello` in variable `b`, then prints `hello`.

This is one of the strongest demo flows because it proves:

- process creation
- memory usage
- swapping
- scheduling
- file I/O

- correct inter-process interaction

---

16. Code Structure Explanation For Teammates

`Main.java`

This is the simulation coordinator.

It handles:

- resetting the simulator
- reading arguments
- choosing the scheduler
- process arrivals
- dispatching processes
- swapping in and out
- printing the system snapshot

`Interpreter.java`

This is the execution engine.

It:

- fetches the next instruction
- decodes it
- runs the correct system call or mutex operation
- updates the PC and burst time

`ProgramLoader.java`

This converts a program file into a process in memory.

It:

- reads all lines
- calculates the required memory size
- allocates a memory region
- stores instructions
- writes the PCB

`Memory.java`

This simulates actual RAM.

It:

- allocates memory
- frees memory
- stores variables
- stores PCB
- fetches instructions
- swaps processes to disk
- restores them later

`PCB.java` and `Process.java`

These hold process-level metadata used by the scheduler and interpreter.

`Scheduler.java`

This decides who runs next.

It includes:

- HRRN logic
- RR logic
- MLFQ logic
- quantum handling
- swap candidate selection

`MutexManager.java`

This is responsible for:

- resource locking
- blocked queues
- unblocking
- ownership transfer after `semSignal`

`SystemCalls.java`

This is the interface to:

- file reading
- file writing
- console output
- input acquisition
- variable memory access

`SimulatorGUI.java`

This is the presentation layer.

It:

- gathers inputs
- launches the simulation
- captures output
- shows summary metrics
- gives evaluation help

---

17. Recommended Speaking Script For Evaluation

You can say:

"This project is a Java simulation of an operating system. Each text file becomes a process when it arrives. The simulator allocates memory for the process, stores its instructions, variables, and PCB, then schedules it using either HRRN or Round Robin. Shared resources are protected using mutexes. If memory becomes full, the simulator swaps one process to disk and restores it later when needed. The GUI helps us run the system and explain the results more clearly."

Then continue with:

"In the trace area, we can follow every important system event: arrival, scheduling, instruction execution, queue changes, memory state, swapping, and process completion."

---

18. Final Takeaway

This project is not just a code submission. It is a full educational simulation of:

- scheduling
- memory management
- synchronization
- process lifecycle

- swapping

The backend proves correctness, and the GUI makes the whole system easier to understand, present, and evaluate.